DPS941 Challenge Lab #1

Nicholas Krilis, Cassandra Laffan, Nathaniel Ngo, Matthew Phan

General Design

The general design of this program, detailed in the source code included in this submission, is as follows: it takes the turning functions from the first and second labs, adds a drive backwards function, a sonar detection function with functionality to stop when something is in front of it, functions which detect lines with the line detectors and functions which pick and and put down items.

The line detectors were taken off at the last minute, as the distance out in front of the robot was hindering its ability to pick up items.

It's autonomous logic includes adjusting its arm depending on how heavy the item it is holding is, as the arm isn't strong enough to lock items in place. Instead, it has to adjust every time the arm goes below a certain value on the potentiometer. It also will stop if something is detected in the sonar, as to avoid collisions. Finally, while not implemented, it also has logic to stay within a box which is outlined in dark tape.

In the C++ code, there is logic which sends individual characters over the connection from the BeagleBone to the Vex; the Vex will then detect which character was sent and execute the command associated with it. The BeagleBone takes these commands over the command line.

Cross compiling was established in Visual Studio and the code will be included in the submission of this lab.

Control Structures

These are defined as the purposeful behaviors that the robot performs. In this program, the goal of our robot is place four balls into the basket at the far side of a small course.

General formula:
S = (A, B, C)

Recalling that:
A = Is the artifact or goal for the robot at a given point.
B = The behavior that governs what the robot will do to achieve the given goal.
C = Fuzzy logic that dictates the behavior following the task at hand, given the current environment.

Control structure 1:
S1=(distance, driveForward(distance), near(object))

Where: distance is the distance that the robot has preprogrammed to drive in a given command, driveForward(distance) is the pseudo coded behavior in which the robot drives forward the specified distance, and finally, near(object) checks if the robot is approaching an object with its sonar, and will stop if its near an object.

Control structure 2:
S2=(nearObject, stop(driveForward), moveAway(object))

Where: nearOject is the detection of an object, stop(driveForward) will halt the robot and prevent it from driving forward, and moveAway(object) is fuzzy logic that tells the robot to move away from the object which is stopping it. Moving away is a command sent by the individual controlling the robot.

Control structure 3:
S3=(turnAngle, turn(turnAngle), near(turnAngle))

Where: turnAngle is the angle that the robot needs to turn in a given command, in this case all of the angles are 90 degrees, turn(turnAngle) dictates how the robot turns the given angle, and near(turnAngle) checks to see if the elapsed turning angle is equal to the angle which it requires to turn.

Control structure 4:
S4=(turnRemaining, turnControl(turnRemaining), near(turnAngle))

Where turnRemaining is the amount of degrees remaining in a given turn, turnControl(turnRemaining) dictates the behavior of the robot as it slows its turn and near(turnAngle) checks, again, if the elapsed turning angle is approaching the max amount of degrees which the robot needs to turn.

Control structure 5:
S5=(armHeight, isHolding(object), adjust(armHeight))

Where armHeight is the measurement of how high the arm is holding an object, isHolding(object) is a boolean value which checks to see if the robot is actually holding something, and adjust(armHeight) is the fuzzy logic which will adjust the arm's height if it lowers beyond a preprogrammed value. This had to be implemented because, at times, objects are heavier than the robot can handle.

Control structure 6:
S6=(executeCommand, detect(command), instruct(command))

Where executeCommand is what the robot needs to do when it receives a command from the BeagleBone, detect(command) is the logic where it pulls the command from the BeagleBone and the instruct(command) is the logic where it detects which command was sent to it and executes it based on the message's contents.